Chapter 01
Introduction to C++

Learning Objectives

- Understand the strengths of C++ and other languages.
- Be introduced to the C++ standard library of reusable components.
- Compile and run a C++ application using our three preferred compilers.
- Be introduced to object-technology concepts used in the objects-natural case studies.

C and C++

- C Programming Language
  - Implemented in 1972 by Dennis Ritchie at Bell Laboratories
  - Widely known as the UNIX operating system's development language.
  - Available for most computers and is hardware independent
  - The American National Standards Institute (ANSI) cooperated with the International Organization for Standardization (ISO) to standardize C worldwide—the joint standard document was published in 1990.

C and C++

- C++ Programming Language
  - Developed by Bjarne Stroustrup in 1979 at Bell Laboratories as an extension of C.
  - Originally called “C with Classes,”
  - Renamed C++ in the early 1980s
    - derives from C’s increment operator
  - Provides many features that “spruce up” C
  - Includes capabilities for object-oriented programming inspired by the Simula simulation programming language.
  - Has evolved over the last 40 years to enable
    - procedural programming,
    - object-oriented programming,
    - functional-style programming,
    - generic programming, and
    - template metaprogramming.

C and C++

- Modern C++
  - Includes the C++23, C++20, C++17, C++14 and C++11 standards
    - the digits in each name indicate the year in the 2000s when that version was approved by the ISO C++ standard committee.
    - make C++ easier to use, improve runtime performance and software quality, and enable applications to take advantage of today's multi-core systems.
    - We emphasize Modern C++ idioms in this course.
      - C++ is over four decades old, so there is much existing C++ legacy code, code using older programming idioms.

C and C++.

- C++ Core Guidelines
  - Bjarne Stroustrup, Herb Sutter (the ISO C++ committee chair) and several other contributors have created the evolving C++ Core Guidelines
    - containing extensive recommendations and sample code to help you use Modern C++ correctly and effectively.
    - Various code-analysis tools enable you to check your code to ensure it follows these recommendations.
    - We will show the C++ Core Guidelines frequently throughout this course.

C++ Standard Library and Open-Source Libraries

- C++ programs consist of pieces called functions and classes.
  - You can program all the pieces you'll need yourself.
  - Instead, most C++ programmers reuse the rich collections of classes and functions in the C++ standard library.
- Thus, there are really two parts to learning C++ programming:
  - learning the C++ language itself (often referred to as the "core language"), and
  - learning how to use the C++ standard library's classes and functions.

C++ Standard Library and Open-Source Libraries

- The Boost Project and Other Open Source Libraries
  - There are enormous numbers of open-source C++ libraries that can help you perform significant tasks by writing only modest amounts of code.
  - The **Boost** C++ libraries provide 168 powerful open-source libraries.
  - Boost serves as a “breeding ground” for new capabilities that might eventually be incorporated into the C++ standard libraries.

C++ Standard Library and Open-Source Libraries.

- The Boost Project and Other Open Source Libraries
  - The following StackOverflow post lists Modern C++ libraries and language features that evolved from the Boost libraries:
    - https://stackoverflow.com/a/8852421
  - On GitHub, developers contribute to almost 56,000 C++ code repositories:
    - https://github.com/topics/cpp
  - The following pages provide curated lists of popular C++ libraries for many application areas:
    - https://github.com/fffaraz/awesome-cpp
    - https://github.com/rigtorp/awesome-modern-cpp

Building-Block Approach to C++ Development

- Throughout this course, we encourage a building-block approach to creating programs.
- When programming in C++, you'll typically use the following building blocks:
  - C++ standard library classes and functions,
  - open-source C++ library classes and functions,
  - functions and classes you create, and
  - classes and functions other people have created and made available to you.

Building-Block Approach to C++ Development.

- Creating your own classes and functions
  - Advantage: knowing exactly how they work
  - Disadvantage: time-consuming effort that goes into designing, developing, debugging and performance-tuning them.
- Using C++ Standard Library classes and functions instead of writing your own versions can
  - improve program performance because they're written carefully to perform efficiently,
  - Shortens program development time, and
  - improves program portability because they're included in every C++ implementation.
- We focus on using the existing C++ standard library and various open-source libraries to leverage your program development efforts and avoid "reinventing the wheel." — This is called software reuse and is a key part of our objects-natural teaching methodology.

Introduction to Object Orientation

- Building software quickly, correctly and economically remains an elusive goal at a time when demands for new and more powerful software are soaring.
- Objects, or more precisely the classes objects come from, are essentially reusable software components.
  - There are date objects, time objects, audio objects, video objects, automobile objects, people objects, etc.
  - Almost any noun can be reasonably represented as a software object in terms of attributes (e.g., name, color and size) and behaviors (e.g., calculating, moving and communicating).
- Software developers have discovered that using a modular, object-oriented design-and-implementation approach can make software development groups much more productive than was possible with earlier techniques—object-oriented programs are often easier to understand, correct and modify.

Introduction to Object Orientation

- Automobile as an Object
  - Suppose you want to drive a car and make it go faster by pressing its accelerator pedal. What must happen before you can do this?
    - Well, someone has to design the car before you can drive it.
  - Begins as engineering drawings, similar to the blueprints that describe the design of a house.
    - These drawings include the design for an accelerator pedal.
  - The pedal hides from the driver the complex mechanisms that make the car go faster, just as the brake pedal hides the mechanisms that slow the car, and the steering wheel hides the mechanisms that turn the car.
    - This enables people with little or no knowledge of how engines, braking and steering mechanisms work to drive a car easily.
  - Before you can drive a car, it must be built from the engineering drawings that describe it.
    - A completed car has an actual accelerator pedal to make the car go faster, but even that's not enough—the car won't accelerate on its own, so the driver must press the pedal to accelerate the car.

Introduction to Object Orientation

- Functions, Member Functions and Classes
  - Let's use our car example to introduce some key object-oriented programming concepts.
  - Performing a task in a program requires a function.
    - Houses the program statements that perform its task.
    - Hides these statements from its user just as the accelerator pedal of a car hides from the driver the mechanisms of making the car go faster.
  - In C++, we often create a program unit called a class to house the set of functions that perform the class's tasks—these are known as the class's member functions.
    - A class representing a bank account might contain a member function to deposit money to an account, another to withdraw money from an account and a third to query the account's current balance.
    - A class is similar to a car's engineering drawings, which house the design of an accelerator pedal, brake pedal, steering wheel, and so on.

Introduction to Object Orientation

- Instantiation
  - Just as someone has to build a car from its engineering drawings before you can drive a car, you must build an object from a class before a program can perform the tasks defined by the class's member functions.
  - The process of doing this is called instantiation, and an object is then referred to as an instance of its class.

Introduction to Object Orientation

- Reuse
  - Just as a car's engineering drawings can be reused many times to build many cars, you can reuse a class many times to build many objects.
  - Reusing existing classes when building new classes and programs saves time and effort.
  - Reuse also helps you build more reliable and effective systems because existing classes and components often have been extensively tested, debugged and performance-tuned.
  - Just as the notion of interchangeable parts was crucial to the Industrial Revolution, reusable classes are crucial to the software revolution spurred by object technology.

Introduction to Object Orientation

- Messages and Member-Function Calls
  - When you drive a car, pressing its gas pedal sends a message to the car to perform a task—that is, to "go faster."
  - Similarly, you send messages to an object. Each message is implemented as a member-function call that tells a member function of the object to perform its task.
    - For example, a program might call a particular bank account object's deposit member function to increase the account's balance by the deposit amount.

Introduction to Object Orientation

- Attributes and Data Members
  - Besides having capabilities to accomplish tasks, a car also has attributes,
    - such as its color, number of doors, amount of gas in its tank, current speed and record of total miles driven (i.e., its odometer reading).
    - Like its capabilities, the car's attributes are represented as part of its design in its engineering diagrams (which, for example, include an odometer and a fuel gauge).
    - As you drive a car, these attributes are "carried along" with the car. Every car maintains its own attributes. For example, each car knows how much gas is in its own gas tank but not how much is in the tanks of other cars.
  - An object, similarly, has attributes that it carries along as it's used in a program. These attributes are specified as part of the object's class as class's data members.
    - For example, a bank-account object has a balance attribute representing the amount of money in the account. Each bank-account object knows the balance in the account it represents but not the balances of the other accounts in the bank.

Introduction to Object Orientation

- Encapsulation
  - Classes encapsulate (i.e., wrap) attributes and member functions into objects created from those classes—an object's attributes and member functions are intimately related.
  - Objects may communicate with one another, but they're normally not allowed to know how other objects are implemented internally.
  - Those details are hidden inside the objects. As we'll see, this information hiding is crucial to good software engineering.

Introduction to Object Orientation

- Inheritance
  - A new class of objects can be created quickly and conveniently by inheritance.
  - The new class absorbs the characteristics of an existing class, possibly customizing them and adding unique characteristics of its own.
  - In our car analogy, an object of the class “convertible” certainly is an object of the more general class “automobile,” but more specifically, the roof can be raised or lowered.

Introduction to Object Orientation

- Object-Oriented Analysis and Design
  - Soon you'll be writing programs in C++. How will you create the code for your programs? Perhaps, like many programmers, you'll turn on your computer and start typing.
    - This approach may work for small programs (like the ones we present in the book's early chapters), but what if you were asked to create a software system to control thousands of automated teller machines for a major bank?
    - Or suppose you were asked to work on a team of thousands of software developers building the next generation of the U.S. air traffic control system?
  - For projects so large and complex, you should not simply sit down and start writing programs.

Introduction to Object Orientation.

- Object-Oriented Analysis and Design.
  - To create the best solutions, you should follow a detailed analysis process for determining your project's requirements (i.e., defining what the system is supposed to do) and developing a design that satisfies them (i.e., deciding how the system should do it).
  - Ideally, you'd go through this process and carefully review the design (and have your design reviewed by other software professionals) before writing any code.
  - If this process involves analyzing and designing your system from an object-oriented point of view, it's called an object-oriented analysis and design (OOAD) process.
  - C++ has object-oriented programming (OOP) capabilities that enable you to implement object-oriented designs as working systems.

Simplified View of a C++ Development Environment

- C++ systems generally consist of three parts: a program development environment, the language and the C++ Standard Library.
- C++ programs typically go through six phases:
  - edit,
  - preprocess,
  - compile,
  - link,
  - load and
  - execute.

Simplified View of a C++ Development Environment

- Phase 1: Editing a C++ Program
  - You type a C++ program's source code using the editor, make any necessary corrections and save the program on your computer's disk. C++ source code filenames often end with the .cpp, .cxx, .cc or .C (uppercase) extensions, which indicate that a file contains C++ source code.
  - Integrated development environments (IDEs) are available from many major software suppliers.
    - IDEs provide tools that support the software development process, including editors for writing and editing programs and debuggers for locating logic errors—errors that cause programs to execute incorrectly.

Simplified View of a C++ Development Environment

- Phase 2: Preprocessing a C++ Program
  - In Phase 2, you give the command to compile the program.
  - In a C++ system, a preprocessor program executes automatically before the compiler's translation phase begins.
    - The C++ preprocessor obeys commands called preprocessing directives, which indicate that certain manipulations are to be performed on the program before compilation.
    - These manipulations usually include (i.e., copy into the program file) other text files to be compiled and perform various text replacements.
    - A detailed discussion of preprocessor features appears later.

Simplified View of a C++ Development Environment

- Phase 3: Compiling a C++ Program
  - In Phase 3, the compiler translates the C++ program into machine-language code—also referred to as object code:

Simplified View of a C++ Development Environment

- Phase 4: Linking to Create an Executable Program
  - Phase 4 is called linking.
  - C++ programs typically contain references to functions and data defined elsewhere, such as in the standard libraries or in the private libraries of groups of programmers working on a particular project.
  - The object code produced by the C++ compiler typically contains “holes” due to these missing parts. A linker links the object code with the code for the missing functions to produce an executable program (with no missing pieces).

Simplified View of a C++ Development Environment

- Phase 5: Loading an Executable Program
  - Phase 5 is called loading. Before a program can be executed, it must first be placed in memory:
  - This is done by the loader, which takes the executable image from disk and transfers it to memory.
  - Additional components from shared libraries that support the program are also loaded.

Simplified View of a C++ Development Environment

- Phase 6: Executing the Program
  - Finally, the computer, under the control of its CPU, executes the program one instruction at a time.
  - Modern multi-core computer architectures can execute several instructions in parallel.

Simplified View of a C++ Development Environment

- Problems That May Occur at Execution Time
  - Programs might not work on the first try.
    - Each of the preceding phases can fail because of various errors.
      - For example, an executing program might try to divide by zero (an illegal operation for integer arithmetic in C++).
      - This would cause the C++ program to display an error message.
      - If this occurred, you'd have to return to the editing phase, make the necessary corrections and proceed through the remaining phases again to determine that the corrections fixed the problem(s).
    - Certain C++ functions take their input from cin (the standard input stream; pronounced "see-in"), which is normally the keyboard, but can be redirected to another device.
    - Data is often output to cout (the standard output stream; pronounced "see-out"), which is normally the computer screen, but can be redirected to another device.
      - When we say that a program prints a result, we normally mean that the result is displayed on a screen. Data may be output to other devices, such as disks, hardcopy printers, or even transmitted over the Internet.
    - There is also a standard error stream referred to as cerr. The stream (normally connected to the screen) displays error messages.

Simplified View of a C++ Development Environment.

- Problems That May Occur at Execution Time
  - Errors such as division by zero occur as a program runs, so they're called runtime errors or execution-time errors.
  - Fatal runtime errors cause programs to terminate immediately without having successfully performed their jobs.
  - Nonfatal runtime errors allow programs to run to completion, often producing incorrect results.

Test-Driving a C++20 Application Various Ways

- We'll show how to compile and execute C++ code using:
  - Microsoft Visual Studio Community Edition for Windows
  - Code::Blocks on Windows
  - GNU g++ in a shell on Linux

Microsoft Visual Studio Community Edition for Windows

- https://visualstudio.microsoft.com/downloads/
- Click **Free Download** under **Community**
  - Click **Run** to start the installation process
  - Otherwise, double-click the installer file in your **Downloads** folder
- In the **User Account Control** dialog, click **Yes**
  - Allows the installer to make changes to your system
- In the **Visual Studio Installer** dialog, click **Continue**
  - Allows the installer to download the components it needs
- Select the option **Desktop Development with C++**
  - Includes the Visual C++ compiler and the C++ standard libraries
- Click **Install**

Microsoft Visual Studio Community Edition for Windows

- Click Build New Project 建立新專案
- Double Click Console Application 主控台應用程式
- Set Project Title and Location
- Click ▸ to build and run

Code::Blocks on Windows

- Install and Download Code::Blocks without Compiler
- Install compiler for supporting the newest C++ standard
  - Download the compiler from the Code:Blocks forum https://forums.codeblocks.org/
    - Check the post of nightly build
  - Extract the zip file to a folder, e.g., C:\mingw64
  - Run command prompt and call gcc --version to check the gcc version

Code::Blocks on Windows

- Set mingw-w64 as default compiler
  - Launch Code::Blocks
  - Click Settings -> Compiler
  - Select GNU GCC Compiler
  - Click Copy and set the name of compiler, e.g., MinGW-W64
  - Select Toolchain executables
  - Set Compiler's installation directory at C:\mingw64\bin
  - Click Compiler settings -> Compiler Flags
  - Add and activate a new flag
    - Right-click the window, click New flag, and set as follows:
    - Name: Have g++ follow the C++20 ISO C++ language standard
    - Compiler flags: -std=c++20
    - Category: General
    - Supersedes: -std=c++98 -std=gnu++98 -std=c++11 -std=gnu++11 -std=c++14 -std=gnu++14 -std=c++17 -std=gnu++17 -std=c++20 -std=gnu++20
    - Exclusive: false
  - Click Set as default to change the default compiler

Code::Blocks on Windows

- Create a new project
  - File -> New -> Project
  - Double-click Console application
  - Select C++ as the language
  - Set Project Title and Location
  - Select Compiler and Finish
  - Click  to build and run

GNU g++ in a shell on Linux

- Take ubuntu as an example
- Open an terminal
- Install gcc by the following commands
  - `sudo apt-get update`
  - `sudo apt-get install gcc-13 g++-13`
  - `sudo update-alternatives --install /usr/bin/gcc gcc /usr/bin/gcc-13 60 --slave /usr/bin/g++ g++ /usr/bin/g++-13`
  - `g++ --version`

GNU g++ in a shell on Linux

- Coding via text editor
- Compile and run the program
  - g++ -std=c++20 -Wall -o main main.cpp
  - ./main